

23. Median in a row-wise sorted Matrix

```
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
    int median(vector<vector<int>> &mat) {
        int n = mat.size();
        if (n == 0) return 0;
        int m = mat[0].size()
        // Value range (given constraints). If unknown, could set:
        // minVal = min element of first column, maxVal = max element of last column.
        int low = 1;
        int high = 2000; // Alternatively to tighten bounds:
        // low = mat[0][0];
        // high = mat[0][m-1];
        // for (int i = 1; i < n; ++i) {
        //     low = min(low, mat[i][0]);
        //     high = max(high, mat[i][m-1]);
        // }
        int desired = (n * m + 1) / 2; // position of median (1-based)
        while (low < high) {
            int mid = low + (high - low) // count numbers <= mid
            long long cnt = 0;
            for (int i = 0; i < n; ++i) {
                cnt += upper_bound(mat[i].begin(), mat[i].end(), mid) - mat[i].begin();
            }
            if (cnt >= desired) {
                high = mid;
            } else {
                low = mid + 1; return low;
            }
        }
    }
};
```

The screenshot shows a coding platform interface with a dark theme. On the left, the 'Output Window' is open, displaying 'Compilation Results' for 'Custom Input' by 'Y.O.G.I. (AI Bot)'. A green checkmark indicates 'Problem Solved Successfully'. Below this, statistics are shown: 'Test Cases Passed: 1117 / 1117', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 8 / 8', and 'Time Taken: 0.57'. At the bottom, there are buttons for 'Solve Next' and 'Stay Ahead With:' followed by links to 'Reverse Spiral Form of Matrix', 'Binary Matrix with at most K 1s', and 'Aggressive Cows'.

The right panel shows the C++ code for the solution. The code is in C++ (17) and includes a 'Start Timer' button. The code implements a binary search algorithm to find the median in a row-wise sorted matrix. It starts by defining the range of values (low and high) and then iteratively narrows down the search range until the median is found. The median is determined by counting the number of elements less than or equal to the current mid value across all rows.